SCALE THE PROVEN

Set Up Your AI Brain
The Plain-English Guide

Every step is a click or a paste, and every part ends with “you know it worked when” — about two hours from nothing to a working AI Brain.

Set Up Your AI Brain — The Plain-English Guide

This guide assumes you know **nothing** about setting this up. Every step tells you exactly what to click or type, and what you should see when it worked. You already know your everyday tools (Outlook, Airtable, Granola, Ahrefs, Canva) — this teaches only the new part.

What you're building, in one sentence: a folder on your computer that acts like a filing cabinet, a team of AI employees who read and write in that cabinet, and a rulebook so nothing goes out the door without your OK.

Total time: about 2 hours, split into 9 parts. You can stop after any part and pick up later.

Part 1 — Install the two programs (20 min)

You need two things: the **Claude desktop app** (your daily driver) and **Claude Code** (the builder tool).

1a. Claude desktop app

1. Go to claude.ai/download.
2. Download the app for your computer and install it like any other program.
3. Open it and sign in with your Claude account.

✅ **You know it worked when:** you can type a message to Claude in the app.

1b. Node.js (a helper Claude Code needs)

1. Go to nodejs.org.
2. Click the big green button that says **LTS** ("Long Term Support").
3. Run the installer. Click **Next** on everything. Don't change any settings.

1c. Claude Code

1. Open your computer's command window: - **Windows:** press the Windows key, type `powershell`, press Enter. - **Mac:** press Cmd+Space, type `terminal`, press Enter. A window with text and a blinking cursor appears. That's it — you type commands here and press Enter. You'll only need it a few times.
2. Type this exactly and press Enter: `npm install -g @anthropic-ai/claude-code`
3. Wait until the text stops scrolling (a minute or two).
4. Type `claude --version` and press Enter.

✅ **You know it worked when:** it prints a version number instead of an error.

Part 2 — Build the filing cabinet (10 min)

1. Open your file manager (File Explorer / Finder).
2. In your **Documents** folder, create a new folder named exactly: `CenseoAI-Brain`
3. Open the command window again (like in step 1c) and type: `cd Documents/CenseoAI-Brain` press Enter, then type: `claude` press Enter. Claude Code starts *inside your new folder* — that matters; it can only see the folder you start it in.
4. Paste this message into Claude Code and press Enter: `Create this folder structure for my AI second brain: context/, inbox/, projects/, people/, daily/, sops/, templates/, archive/, .claude/skills/, .claude/agents/. Add a short README.md in each folder explaining what goes in it. Also create an empty memory.md in the root.`

✅ **You know it worked when:** you look at the folder in File Explorer and see all those subfolders. (The `.claude` folder may be hidden — that's normal.)

Part 3 — Hire your team (10 min)

Your skills (the jobs) and agents (the employees) are already written and stored in your GitHub repo. Instead of copying files by hand, ask Claude to do it. Paste this into Claude Code:

```
Clone https://github.com/censeotc/claude-code-starter into a temporary
folder. Copy every folder from its docs/ai-brain-skills/ into my
.claude/skills/, and every .md file from its docs/ai-brain-agents/ into
my .claude/agents/. Then delete the temporary folder and list what you
installed.
```

✅ **You know it worked when:** Claude lists 10 skills (inbox-triage, client-brief, meeting-prep, follow-up, braindump, sop-writer, deal-prep, revenue-scoreboard, client-report, aeo-snapshot) and 12 agents (revops-engineer, finance-manager, account-manager, and so on).

Now close Claude Code (type `/exit`) and start it again (type `claude`). It re-reads the folder on startup — do this restart any time you add skills or agents.

Part 4 — Teach it your business (45 min, the most important part)

This is what makes it *your* brain instead of a generic robot. Have your **Business Model Canvas open** next to you — it answers most of the questions Claude will ask.

4a. The master briefing

Paste into Claude Code:

```
I'm building my personal AI Brain for my business, CenseoAI. Interview me,
one question at a time, then write a CLAUDE.md file for this vault covering:
who I am and my role, what CenseoAI does and sells (the service ladder),
who my customers and buyers are, my tone of voice, and how I want you to
respond (concise, numbers first, no fluff). Include a Vault Conventions
section explaining each folder, and these two standing rules:
1) Read memory.md at the start of every session and update it when I make
   a decision or commitment.
2) Never send an email, create a calendar event, or take any external
   action without showing me a draft and getting my explicit OK.
```

Answer its questions like you're explaining to a new employee on day one. Real numbers, real client names, real prices.

4b. The context files

Paste this next:

```
Now interview me one file at a time and write these notes in context/:
- business.md — the offer: Revenue Leakage Audit -> 45-Day Pilot ->
  90-Day Revenue Operating System -> optimization retainer, with real
  prices; the no-rip-and-replace promise; the tools we build on
- icp.md — primary: HVAC home services, 10+ trucks, service-heavy, on
  ServiceTitan; secondary trades; the four buyers (Owner, GM/COO,
  Ops/Service Manager, Sales Manager) and what each one cares about
- brand.md — my tone of voice, with 3 real writing samples I'll paste
- strategy.md — my goals for this quarter and the numbers that matter
- team.md — me plus the contractor bench and what each contractor owns
- operator.md — my strengths, my weaknesses, and what I never want to
  do manually again
```

For `brand.md`: paste in three real emails you wrote that sound like you — including one where you said no to something. That's what makes drafts sound like Scott.

4c. The test

Close Claude Code (`/exit`), reopen it (`claude`), and ask:

```
What do you know about my business?
```

✅ **You know it worked when:** the answer sounds like someone who works at CenseoAI. If it's generic, the context files are too thin — go add what's missing. Repeat until it passes.

Part 5 — Plug in your tools (15 min)

Your daily tools connect through the **Claude desktop app**, and many may already be connected from your existing setup.

1. First, check what's already there. In the desktop app, ask: `List the MCP tools available to you.`
2. In the desktop app: **Settings** → **Connectors**. Connect anything from this list that's missing: - **Microsoft 365** (your Outlook email + calendar — the important one) - **Gmail** (the second inbox) - **Google Calendar** and **Google Drive** - **GoHighLevel** (your CRM — the system of record for contacts, deals, and pipelines; connect via its API/MCP server. Airtable only if you still use it for non-CRM databases) - **Granola** (meeting transcripts) - **Ahrefs** (SEO/AEO data) - **Canva** (branded deliverables)
3. Each one opens a login window — sign in with that tool's account and click Allow.
4. **Fully quit and reopen the app** (connections load at startup only).
5. Ask again: "List the MCP tools available to you."

✅ **You know it worked when:** you see tools named after Outlook, Gmail, Airtable, and the rest in the list.

Part 6 — Lock the doors (10 min)

Two layers, because a promise plus a lock beats a promise.

1. **The promise** — already in your CLAUDE.md from Part 4a (rule 2).
2. **The lock** — in Claude Code, type `/permissions` and press Enter. Find the tools that *send* or *create* things in the outside world (send email, create calendar event) and set them to **Ask**. Now the software itself will stop and ask you every time, no matter what any prompt says.

✅ **You know it worked when:** you ask Claude to "send a test email to yourself" and it shows you a draft and an approval prompt instead of just sending.

6c. Turn on the black box recorder (governance + logging)

Your governance framework (the CenseoAI Governance Template v4.9) and a full audit trail install with one paste. In Claude Code:

```
From https://github.com/censeotc/claude-code-starter, copy
docs/ai-brain-governance/governance-operational.md into my context/ folder
as governance.md. Create a logs/ folder with an empty releases.md and
events.md, and an empty incidents.md in the vault root using the entry
template from docs/ai-brain-governance/incident-playbook.md. Then add the
"Governance & logging" standing rules block from
docs/ai-brain-governance/logging.md to my CLAUDE.md.
```

✅ **You know it worked when:** you ask Claude to do anything, and afterward `logs/` contains today's file with a one-line record of what it did — including the reliance tier. Full details: the logging spec and incident playbook.

Part 7 — Test drive (15 min)

Try one of each, in the desktop app or Claude Code:

1. **A skill:** type `/inbox-triage`. It should sweep Outlook + Gmail and show you sorted piles with drafts. Approve one draft — check it lands in your Drafts folder, not Sent.
2. **An agent:** type "Poke holes in my plan to [something you're actually considering]." The thinking-partner agent should push back on you — that's its job.
3. **The memory:** say "I've decided to [any real small decision]." Then check `memory.md` — the decision should be logged with today's date.

✅ **You know it worked when:** all three behave as described. If a skill doesn't trigger, you probably started Claude Code outside the CenseoAI-Brain folder — close it, `cd Documents/CenseoAI-Brain`, start again.

Part 8 — Make it automatic (10 min)

Right now you have to remember to run things. Schedules fix that. Using Claude Code on the web (claude.ai/code) or the desktop app, create three scheduled tasks (called Routines) by asking in plain English:

```
Create four recurring scheduled tasks for me:
1. Weekdays at 7:00am my time: run /daily-brief and notify me when ready.
2. Mondays at 8:00am: run /follow-up and flag anyone overdue.
3. Fridays at 4:00pm: run /weekly-review with the focus-coach score.
4. Daily at 6:00pm: commit the vault to git as a daily checkpoint and
   verify today's action log has no gaps.
```

Your Part 6 lock still applies — a scheduled run can *prepare* everything but *send* nothing.

✅ **You know it worked when:** the next weekday morning, your brief is waiting for you before you asked.

Part 9 — The four-week ramp (so you actually keep using it)

Don't try to use everything at once. This order builds the habit first and adds weight gradually — the full detail lives in the Orchestration Plan.

Week	Add this, nothing else
1	The daily loop only: read the 7am brief, run /inbox-triage, use /meeting-prep and /meeting-debrief around every meeting
2	The delivery spine on ONE real client: revops-engineer + /revenue-scoreboard + the weekly cadence call
3	The weekly layer: Monday follow-up routine, Friday review, let focus-coach keep the commitment ledger
4	Growth + back office: /deal-prep on a live prospect, first monthly finance close, first client-health pass

After week 4, add one playbook at a time from the Orchestration Plan as real situations trigger them.

When something breaks (keep this handy)

Problem	Fix
Skills don't trigger (`/inbox-triage` does nothing)	You started Claude Code in the wrong folder. Close it, `cd Documents/CenseoAI-Brain`, run `claude` again.
A connected tool's data doesn't show up	Fully quit and reopen the app — connections only load at startup.
Answers feel generic	Context files are thin. Ask "what do you know about my business?", add whatever it couldn't answer.
Email drafts don't sound like you	Add 2–3 more real samples to `context/brand.md`, including one where you say no.
An agent isn't stepping in when expected	Open its file in `.claude/agents/` and make the `description` line mention the exact words you tend to use.
You're worried something sent without asking	It can't, if Part 6 is done — verify with the test-email check again.

The golden rules

1. **Nothing sends without you.** Ever. That's Parts 4a and 6 working together.
2. **When in doubt, ask Claude to do the technical thing.** "Move these files," "fix this setting," "why didn't that work" — it's sitting in your vault and can fix its own house.
3. **Feed the memory.** The system gets smarter every week only if decisions land in memory.md and meeting outcomes get debriefed. The flywheel is the whole product.
4. **Back it up.** Once a week, ask Claude Code: "Commit everything in this vault to git with a summary of what changed." Plain files + git = you can never lose your brain.

Companion documents: the build guide (the why behind each piece) · Orchestration Plan (how the team runs together) · skills · agents